BÀI GIẢNG HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

CHUYÊN ĐỀ: DANH SÁCH LIÊN KẾT ĐƠN

TỔNG QUAN NỘI DUNG BÀI HỌC

1. Ý tưởng – Giải pháp
2. Kiến trúc danh sách liên kết đơn
3. Các bước xây dựng
4. Các thao tác
5. Bài tập

---

1. Ý Tưởng – Giải Pháp

1.1. Vấn đề 1: Lưu trữ một số lượng nhỏ các giá trị

Bài toán đặt ra: Muốn lưu trữ các số: `10`, `79`, `39`, `66`, `26`, `50` thì chúng ta làm sao?
Một cách tiếp cận đơn giản là khai báo các biến rời rạc tương ứng cho từng giá trị:

```
10  79  39  66  26  50
x   y   z   t   g   h
```

1.2. Vấn đề 2: Lưu trữ nhiều số hơn

Bài toán đặt ra: Muốn lưu trữ nhiều số hơn nữa thì làm sao?
Giải pháp quen thuộc là sử dụng **mảng một chiều** (ví dụ cấp phát mảng gồm 500 phần tử có chỉ số từ `0` đến `499`):

index	0	1	2	3	4	5	6	...	499
value	10	79	39	66	26	50		...	

Bảng 1: Minh họa lưu trữ dãy số bằng mảng một chiều.

Câu hỏi mở: *Có giải pháp nào tốt hơn nữa không?* (Việc cấp phát mảng cố định kích thước có thể gây dư thừa vùng nhớ chưa dùng đến hoặc không đủ chỗ khi số lượng phần tử tăng thêm $\Rightarrow$ cần cấu trúc dữ liệu cấp phát linh hoạt theo từng phần tử là **Danh sách liên kết đơn**).

2. Kiến Trúc Danh Sách Liên Kết Đơn

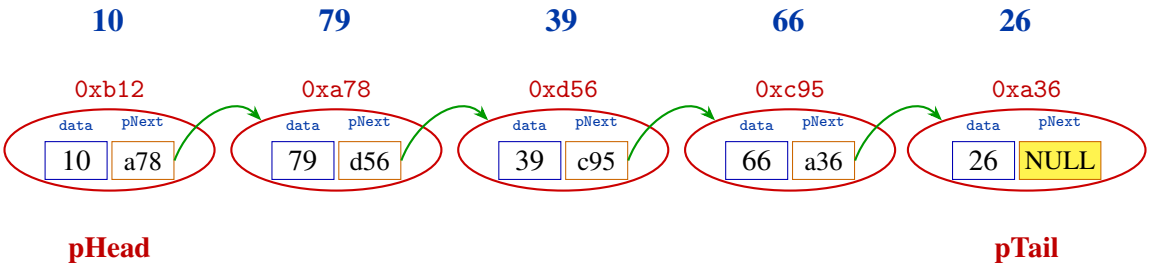
Xét nhu cầu lưu trữ dãy các số nguyên: `10`, `79`, `39`, `66`, `26`.

Thay vì lưu các phần tử liên tiếp nhau trong bộ nhớ như mảng một chiều, mỗi phần tử trong **danh sách liên kết đơn** được đóng gói thành một **Node** gồm hai thành phần:

- `data`: Lưu trữ dữ liệu (giá trị) của phần tử.
- `pNext`: Con trỏ lưu địa chỉ của `Node` kế tiếp trong danh sách (Node cuối cùng trỏ tới `NULL`).

Toàn bộ danh sách được quản lý thông qua hai con trỏ đặc biệt:

- `pHead`: Con trỏ trỏ đến `Node` đầu tiên của danh sách.
- `pTail`: Con trỏ trỏ đến `Node` cuối cùng của danh sách.



Hình 1: Kiến trúc danh sách liên kết đơn trong bộ nhớ.

3. Các Bước Xây Dựng Danh Sách Liên Kết Đơn

3.1. Quy trình 4 bước xây dựng

1. **Bước #1: Tạo cấu trúc Node**

   - Mỗi `Node` chứa dữ liệu (`int data`) và con trỏ trỏ tới Node kế tiếp (`Node* pNext`).

```
struct Node{
    int data;
    Node* pNext;
};
```

2. **Bước #2: Khởi tạo Node từ** `value` *(Viết hàm chuyển `value` thành `Node`)*

   - Cấp phát động một `Node` mới, gán `data = value`, con trỏ `pNext = NULL` và trả về địa chỉ `Node` vừa tạo (ví dụ: Node tại `0xb12` có `data = 10`, `pNext = NULL`).

```
Node* initNode(int value)
{
    Node* p = new Node;

    p->data = value;
    p->pNext = NULL;
    return p;
}
```

3. **Bước #3: Tạo cấu trúc `List`**

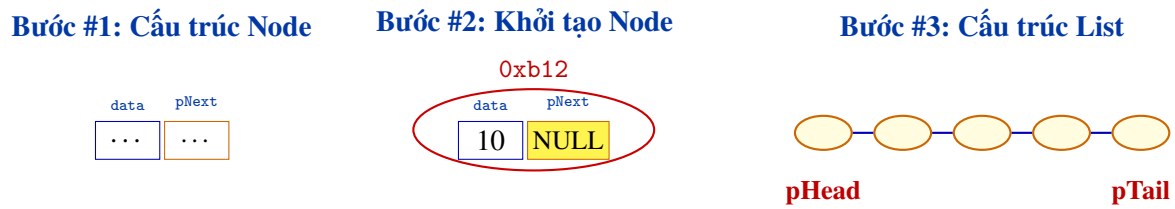
   - Cấu trúc `List` quản lý danh sách thông qua hai con trỏ `pHead` và `pTail`.

```
struct List
{
    Node* pHead;
    Node* pTail;
};
```

4. **Bước #4: Khởi tạo `List`**

   - Khi danh sách mới tạo chưa có phần tử nào (danh sách rỗng), cả `pHead` và `pTail` đều trỏ về `NULL`.

```
void initList(List& l)
{
    l.pHead = NULL;
    l.pTail = NULL;
}
```



Hình 2: Minh họa trực quan các bước cấu trúc Node và List.

3.2. Khai báo – Khởi tạo Node và List

- **Khai báo danh sách và khởi tạo danh sách:**

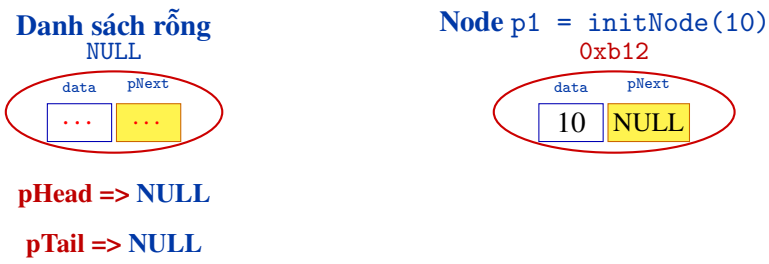
```
List l;
initList(l);
```

Lúc này danh sách rỗng: `pHead => NULL`, `pTail => NULL`.

- **Khai báo Node và khởi tạo Node:**

```
Node* p1 = initNode(10);
Node* p2 = initNode(79);
Node* p3 = initNode(39);
```

Mỗi lần gọi `initNode`, một vùng nhớ động cho `Node` được cấp phát độc lập (ví dụ `p1` tại `0xb12` có `data = 10`, `pNext = NULL`).



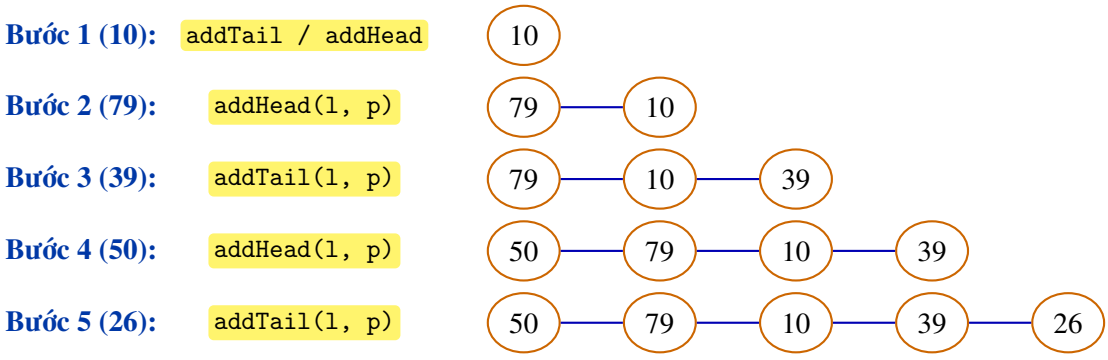
Hình 3: Trạng thái sau khi khởi tạo danh sách rỗng và khởi tạo một Node.

4. Các Thao Tác Trên Danh Sách Liên Kết Đơn

4.1. Tổng quan quá trình thêm Node vào List

Xét dãy giá trị cần đưa vào danh sách theo thứ tự: 10, 79, 39, 50, 26. Với mỗi giá trị, ta tạo một Node mới bằng `Node* p = initNode(...);` rồi gọi `addHead(l, p)` (thêm vào đầu) hoặc `addTail(l, p)` (thêm vào cuối):

1. Giá trị 10: Gọi `addTail(l, p);` hoặc `addHead(l, p);` (vì danh sách đang rỗng) ⇒ `(10)`.
2. Giá trị 79: Gọi `addHead(l, p);` ⇒ `(79) - (10)`.
3. Giá trị 39: Gọi `addTail(l, p);` ⇒ `(79) - (10) - (39)`.
4. Giá trị 50: Gọi `addHead(l, p);` ⇒ `(50) - (79) - (10) - (39)`.
5. Giá trị 26: Gọi `addTail(l, p);` ⇒ `(50) - (79) - (10) - (39) - (26)`.



Hình 4: Diễn biến danh sách qua từng thao tác `addHead` và `addTail`.

4.2. Thêm Node tại vị trí đầu List (addHead)

Hàm `addHead(List& l, Node* p)` được cài đặt theo hai trường hợp: danh sách đang rỗng và danh sách đã có ít nhất một phần tử:

```
void addHead(List& l, Node* p)
{
    if(l.pTail == NULL)
    {
        l.pHead = p;
        l.pTail = p;
    }
    else
    {
        p->pNext = l.pHead;
        l.pHead = p;
    }
}
```

(Ghi chú: Điều kiện `l.pTail == NULL` dùng để kiểm tra danh sách rỗng, tương đương với kiểm tra `l.pHead == NULL`.)

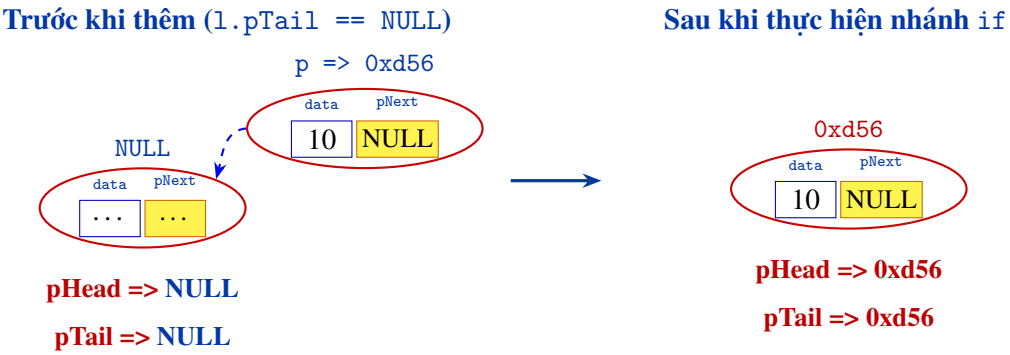
Minh họa từng bước với dãy giá trị `10, 79, 39`:

Trường hợp 1: Danh sách đang rỗng (thêm giá trị `10`)

1. Ban đầu danh sách rỗng (`pHead => NULL`, `pTail => NULL`). Khởi tạo Node `p` tại địa chỉ `0xd56` bằng `initNode(10)` (`data = 10`, `pNext = NULL`).
2. Do `l.pTail == NULL`, thực hiện nhánh `if`:

   `l.pHead = p;  l.pTail = p;`

3. Kết quả: Cả `pHead` và `pTail` cùng trỏ tới `0xd56`.

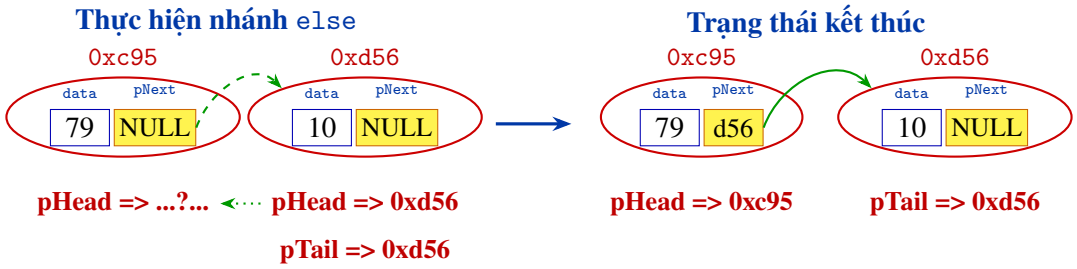


Hình 5: Minh họa `addHead` khi danh sách rỗng (thêm `10` tại `0xd56`).

Trường hợp 2: Danh sách đã có phần tử (tiếp tục thêm giá trị `79`)

1. Danh sách hiện tại có Node `0xd56` (`data = 10`, `pNext = NULL`) với `pHead => 0xd56`, `pTail => 0xd56`.

2. Tạo Node mới `p = initNode(79);` tại địa chỉ `0xc95` (`data = 79`, `pNext = NULL`) rồi gọi `addHead(l, p);`.
3. Vì `l.pTail != NULL`, thực hiện nhánh `else`:
   - `p->pNext = l.pHead;`: Gán địa chỉ pHead hiện tại (`0xd56`) vào trường `pNext` của Node p (`0xc95`).
   - `l.pHead = p;`: Cập nhật pHead trỏ về Node mới `0xc95`.
4. Kết quả: `pHead => 0xc95`, `pTail => 0xd56`; Node `0xc95` nối tới Node `0xd56`.



Hình 6: Minh họa addHead khi danh sách đã có phần tử (thêm 79 tại `0xc95`).

4.3. Thêm Node tại vị trí cuối List (`addTail`)

Hàm `addTail(List& l, Node* p)` được cài đặt tương ứng với hai trường hợp:

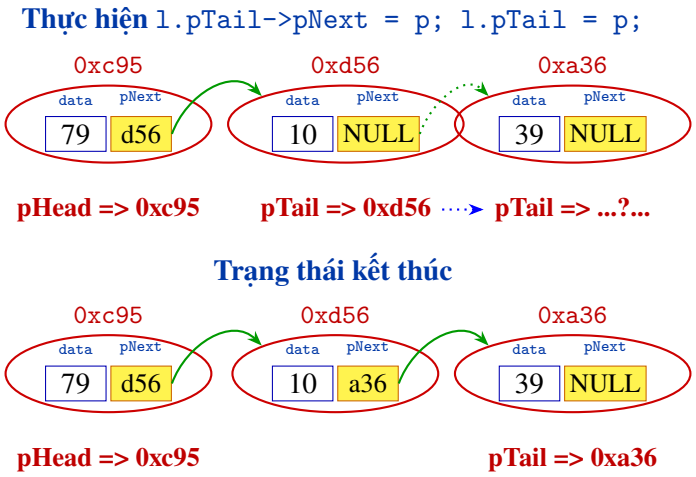
```
void addTail(List& l, Node* p)
{
    if(l.pHead == NULL)
    {
        l.pHead = p;
        l.pTail = p;
    }
    else
    {
        l.pTail->pNext = p;
        l.pTail = p;
    }
}
```

Minh họa từng bước (tiếp tục thêm giá trị `39` vào cuối danh sách hiện có):

1. Danh sách hiện tại có 2 Node: `0xc95 (79, d56)` → `0xd56 (10, NULL)`, với `pHead => 0xc95` và `pTail => 0xd56`.
2. Khởi tạo Node mới `p = initNode(39);` tại địa chỉ `0xa36` (`data = 39`, `pNext = NULL`) và gọi `addTail(l, p);`.
3. Vì `l.pHead != NULL`, thực hiện nhánh `else`:
   - `l.pTail->pNext = p;`: Cho trường `pNext` của Node cuối hiện tại (`0xd56`) lưu địa chỉ a36 để liên kết tới Node mới `0xa36`.

- `l.pTail = p;`: Cập nhật `pTail` chuyển từ `0xd56` sang `0xa36`.

4. Kết quả: `pHead => 0xc95`, `pTail => 0xa36`; danh sách hoàn chỉnh gồm 3 Node: `0xc95 (79)` → `0xd56 (10)` → `0xa36 (39)`.



Hình 7: Minh họa `addTail` thêm Node 39 tại `0xa36` vào cuối danh sách.

4.4. In danh sách (`printList` và `printWh`)

Để duyệt và xuất toàn bộ giá trị của danh sách (ví dụ danh sách `0xc95 (79)` → `0xd56 (10)` → `0xa36 (39)`), ta sử dụng một con trỏ `p` xuất phát từ `l.pHead` và di chuyển lần lượt theo `p = p->pNext` cho đến khi gặp `NULL`.

- **Cách 1: Duyệt bằng vòng lặp `for`**

```
void printList(List l)
{
    for(Node* p = l.pHead; p != NULL; p = p->pNext)
    {
        cout << p->data << "\t";
    }
}
```

- **Cách 2: Duyệt bằng vòng lặp `while`**

```
void printWh(List l)
{
    Node* p = l.pHead;
    while(p!=NULL)
    {
        cout << p->data << "\t";
        p = p->pNext;
    }
}
```

4.5. Viết hàm `main` minh họa

```
int main()
{
    List l;
    initList(l);

    Node* p = initNode(10);

    addHead(l, p);
    addHead(l, initNode(79));
    addTail(l, initNode(39));

    cout << "In danh sach:\n";
    printList(l); //79  10  39

    cout << endl;
    printWh(l);
    return 0;
}
```

Kết quả xuất ra màn hình:

```
In danh sach:
79      10      39
79      10      39
```

4.6. Tìm kiếm trên danh sách liên kết đơn

4.6.1. Tìm kiếm một giá trị có trong List hay không

- **Yêu cầu:** Viết hàm tìm kiếm một giá trị có trong List hay không?
- **Phân tích:**
  - *Input:* List ⇒ `List l`; Giá trị cần tìm ⇒ `int value`.
  - *Output:* True/false ⇒ `return bool`.
- **Mở rộng:** Trả về địa chỉ (`Node*`).

Khung cài đặt trên bài giảng:

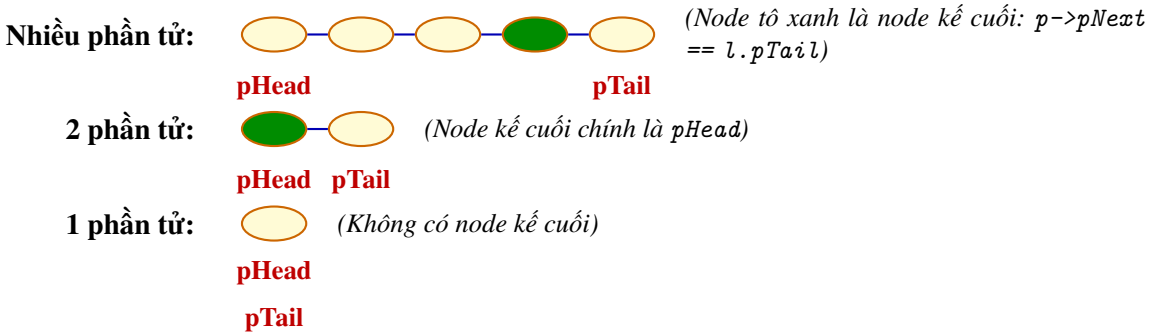
```
bool timGiaTri(List l, int value)
{
    for(Node* p = l.pHead; p != NULL; p = p->pNext)
    {
        if(...........)
        {
            return true;
        }
    }

    return false;
}
```

Gợi ý hoàn thiện: Thay `. . . . . .` bằng điều kiện `p->data == value`. Với phần mở rộng trả về địa chỉ, kiểu trả về của hàm là `Node*`; khi tìm thấy thì `return p;`, nếu duyệt hết danh sách không thấy thì `return NULL;`.

4.6.2. Tìm giá trị Node kế cuối

- **Yêu cầu:** Viết hàm tìm giá trị node kế cuối?
- **Phân tích:**
  - *Input:* List ⇒ `List l`.
  - *Output:* Có hay không ⇒ `return bool`; Giá trị tìm thấy ⇒ giá trị.
- **Mở rộng:** Trả về địa chỉ.



Hình 8: Các trường hợp cần xét khi tìm Node kế cuối trong danh sách.

4.7. Kỹ thuật đếm

- **Yêu cầu:** Viết hàm đếm các giá trị chẵn có trong List?
- **Phân tích:**
  - *Input:* List ⇒ `List l`.
  - *Output:* Số lượng phần tử chẵn ⇒ `return int`.

Hướng thực hiện: Khởi tạo biến đếm `int dem = 0;`, duyệt từ `l.pHead` đến `NULL`; với mỗi Node `p`, nếu `p->data % 2 == 0` thì tăng `dem` lên 1, cuối cùng `return dem;`.

4.8. Tìm giá trị lớn nhất – nhỏ nhất

4.8.1. Tìm giá trị lớn nhất

- **Yêu cầu:** Viết hàm tìm giá trị lớn nhất trong List?
- **Phân tích:**
  - *Input:* List ⇒ `List l`.
  - *Output:* Giá trị lớn nhất ⇒ `return` giá trị.

Khung cài đặt trên bài giảng:

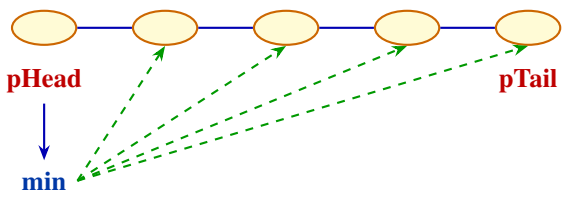
```
int timGiaTriMax(List l)
{
    int m = ..giá trị đầu tiên...;
    for(Node* p = l.pHead; p != NULL; p = p->pNext)
    {
        if(............)
        {
            ............
        }
    }

    return m;
}
```

Gợi ý hoàn thiện: Khởi tạo `int m = l.pHead->data;` (khi danh sách không rỗng); trong vòng lặp kiểm tra `if (p->data > m)` thì cập nhật `m = p->data;`.

4.8.2. Tìm giá trị nhỏ nhất

- **Yêu cầu:** Viết hàm tìm giá trị nhỏ nhất trong List?
- **Phân tích:**
  - *Input:* List ⇒ `List l`.
  - *Output:* Giá trị nhỏ nhất ⇒ `return` giá trị *(ghi chú: slide gốc ghi nhầm chữ "Giá trị lớn nhất").*
- **Kết luận:** ⇒ **Phương pháp tương tự tìm max** (giả sử `min` ban đầu là giá trị của `pHead`, sau đó duyệt qua từng Node đến `pTail` để so sánh và cập nhật).



Hình 9: Minh họa phương pháp tìm giá trị nhỏ nhất (khởi tạo từ `pHead` và duyệt đến `pTail`).

4.9. Tính tổng các giá trị trong List

- **Yêu cầu:** Viết hàm tính tổng các giá trị trong List?
- **Phân tích:**
  - *Input:* List ⇒ `List`.
  - *Output:* Tổng các giá trị ⇒ `return` giá trị.

Khung cài đặt trên bài giảng:

```
int tinh(List l)
{
    int t = 0;
    for(Node* p = l.pHead; p != NULL; p = p->pNext)
    {
        t = //công thức tính tổng
    }

    return t;
}
```

Gợi ý hoàn thiện: Công thức tính tổng là `t = t + p->data;`.

4.10. Xóa Node tại vị trí đầu List (`removeHead`)

- **Yêu cầu:** Viết hàm xóa Node đầu tiên trong List?
- **Phân tích:**
  - *Input:* List ⇒ `List`.
  - *Output:*
    - List đã xóa ⇒ `List&`
    - Kết quả xóa ⇒ `bool`
    - Giá trị đã xóa ⇒ *(trả về qua hàm hoặc tham chiếu)*

Khung hàm trên bài giảng:

```
int removeHead(List& l)
{


    return value;
}
```

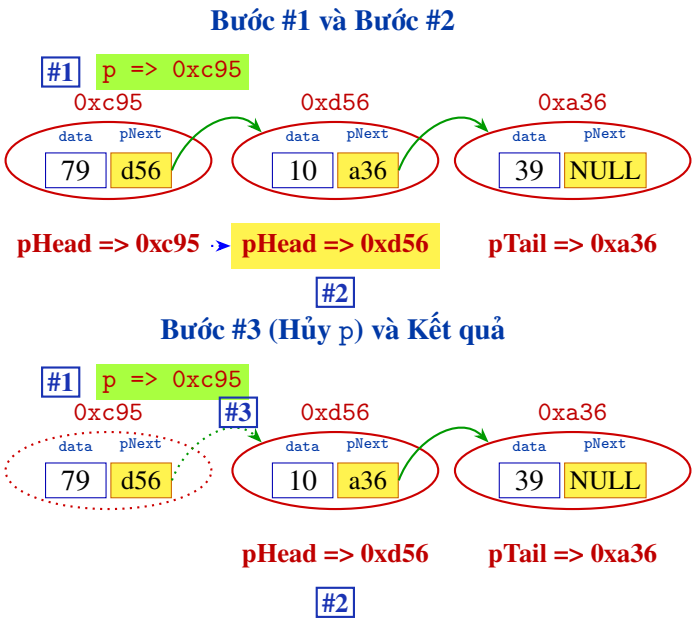
Minh họa từng bước thao tác xóa đầu:

Xét danh sách ban đầu có 3 Node: `0xc95 (79, d56)` → `0xd56 (10, a36)` → `0xa36 (39, NULL)`, với `pHead => 0xc95`, `pTail => 0xa36`.

1. **Bước #1:** Dùng con trỏ tạm `p` trỏ tới Node đầu tiên đang cần xóa (`p => 0xc95`, tương ứng `Node* p = l.pHead;` và lưu lại `int value = p->data;`).

2. **Bước #2:** Di chuyển pHead sang Node kế tiếp (pHead từ 0xc95 sang 0xd56, tương ứng `l.pHead = l.pHead->pNext;`).
3. **Bước #3:** Hủy (thu hồi vùng nhớ) Node mà p đang trỏ tới (`delete p;`).

(Lưu ý: Nếu danh sách trước khi xóa chỉ có 1 Node thì sau bước #2, `l.pHead == NULL`, cần gán thêm `l.pTail = NULL;` để tránh lạc con trỏ.)



Hình 10: Minh họa từng bước xóa Node đầu danh sách (removeHead).

4.11. Xóa Node tại vị trí cuối List (removeTail)

- **Yêu cầu:** Viết hàm xóa Node cuối trong List?
- **Phân tích:**
  - *Input:* List ⇒ List.
  - *Output:*
    - List đã xóa ⇒ List&
    - Giá trị đã xóa ⇒ return

Khung hàm trên bài giảng:

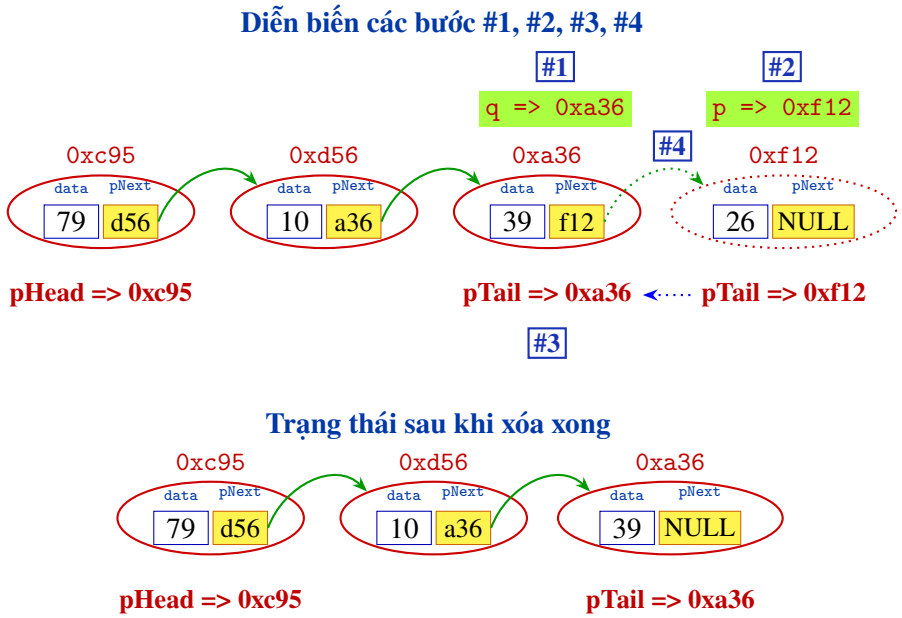
```
int removeTail(List& l)
{


    return value;
}
```

Minh họa từng bước thao tác xóa cuối:

Xét danh sách ban đầu có 4 Node: 0xc95 (79, d56) → 0xd56 (10, a36) → 0xa36 (39, f12) → 0xf12 (26, NULL), với pHead => 0xc95, pTail => 0xf12.

1. **Bước #1:** Tìm Node kế cuối (Node đứng ngay trước `pTail`) và giữ bằng con trỏ `q` (trong hình là `0xa36`, thỏa `q->pNext == l.pTail`).
2. **Bước #2:** Dùng con trỏ `p` trỏ tới Node cuối cùng cần xóa (`p => 0xf12`, tương ứng `Node* p = l.pTail;` và lưu `int value = p->data;`).
3. **Bước #3:** Cập nhật `pTail` trỏ về Node kế cuối `q` (`pTail` từ `0xf12` chuyển sang `0xa36`, tương ứng `l.pTail = q;`).
4. **Bước #4:** Ngắt liên kết tới Node cuối cũ (`l.pTail->pNext = NULL;`) và thu hồi vùng nhớ của Node `p` (`delete p;`).



Hình 11: Minh họa từng bước xóa Node cuối danh sách (`removeTail`).

5. Hệ Thống Bài Tập Ứng Dụng

5.1. Bài Tập 1: Danh sách liên kết đơn quản lý các số nguyên

Đề bài: Viết chương trình cài đặt danh sách liên kết đơn quản lý các số nguyên. Thực hiện theo yêu cầu sau:

1. Xây dựng cấu trúc node và cấu trúc danh sách liên kết đơn.
2. Viết hàm khởi tạo danh sách liên kết đơn và hàm khởi tạo địa chỉ node từ một số nguyên.
3. Viết hàm chèn node vào đầu danh sách.
4. Viết hàm chèn node vào cuối danh sách.
5. Viết hàm khởi tạo giá trị cho danh sách từ mảng một chiều.
6. Viết hàm in giá trị danh sách đã nhập.
7. Viết hàm in giá trị kèm địa chỉ của từng node trong danh sách.
8. Viết hàm kiểm tra danh sách có rỗng hay không.

9. Viết hàm trả về node thứ *n* trong danh sách.
10. Viết hàm tìm kiếm một node có giá trị *X* trong danh sách.
11. Viết hàm tìm kiếm các địa chỉ node có giá trị *X* trong danh sách.
12. Viết hàm đếm số lượng các node có giá trị âm, dương.
13. Viết hàm tìm node có giá trị âm lớn nhất, tìm node có giá trị lẻ nhỏ nhất trong danh sách.
14. Viết hàm cập nhật giá trị cho một node có giá trị là *X* bằng giá trị *Y* mới.
15. Viết hàm copy danh sách.

5.2. Bài Tập 2: Danh sách liên kết đơn quản lý sinh viên

Đề bài: Viết chương trình cài đặt danh sách liên kết đơn quản lý sinh viên với các thao tác như sau:

1. Xây dựng cấu trúc dữ liệu có tên `SinhVien` lưu trữ các thông tin sau: `maSV`, `hoTen`, `diemTB`, `xepLoai`... (hoặc lưu trữ thêm các thông tin khác nếu muốn).
2. Xây dựng cấu trúc node để lưu trữ sinh viên và cấu trúc danh sách liên kết đơn quản lý sinh viên.
3. Viết hàm khởi tạo danh sách liên kết đơn và hàm khởi tạo địa chỉ node từ kiểu `SinhVien`.
4. Viết hàm thêm sinh viên vào đầu danh sách.
5. Viết hàm thêm sinh viên vào cuối danh sách.
6. Viết hàm khởi tạo dữ liệu cho danh sách sinh viên *(Dữ liệu kiểm thử xem cuối bài)*.
7. Viết hàm in danh sách sinh viên.
8. Viết hàm cập nhật xếp loại học lực.
9. Viết hàm tìm điểm trung bình cao nhất.
10. Viết hàm tìm kiếm sinh viên theo mã sinh viên.
11. Viết hàm tìm SV có trung bình cao nhất.
12. Viết hàm tìm SV có điểm trung bình thấp nhất.

Dữ liệu Sinh Viên dùng để kiểm thử:

```
{123, "Nguyen Van A", 9.1, ""};
{124, "Nguyen Van B", 8.8, ""};
{125, "Nguyen Van C", 9.1, ""};
{126, "Nguyen Van D", 2.1, ""};
{127, "Nguyen Van F", 9.7, ""};
{128, "Nguyen Van G", 4.1, ""};
{129, "Nguyen Van H", 8.3, ""};
{130, "Nguyen Van K", 7.9, ""};
```